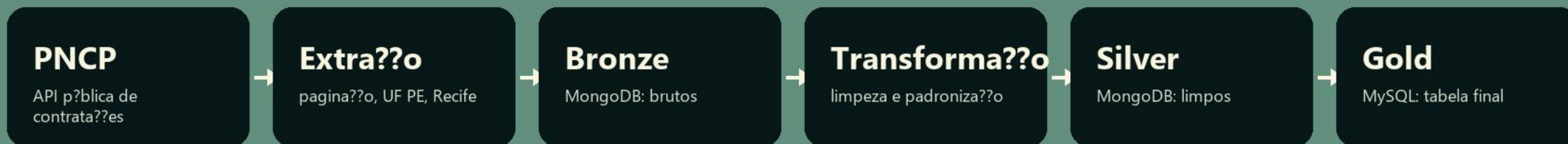


FLUXO ETL DE PONTA A PONTA

O Prefect coordena a coleta no PNCP, a transformação e a persistência em MongoDB e MySQL.



orchestrate_prefect.py

```
201: dados_brutos = task_extrair(...)
211: task_carregar_mongodb(...)
213: registros_limpos = task_transformar(...)
220: task_carregar_mongodb_limpos(...)
226: task_carregar_mysql(...)
```

Fluxo canônico: orchestrate_prefect.py:175

Por que isso importa?

O ETL deixa de ser uma sequência manual de scripts e vira um processo rastreável: cada etapa tem logs, retries e ordem clara de execução.

CAMADAS BRONZE, SILVER E GOLD

A persistência hierárquica separa rastreabilidade, dado limpo e consumo analítico.

BRONZE | MongoDB

Coleção `contratacoes_brutas`. Guarda o documento completo vindo do PNCP, útil para auditoria e reprocessamento.

Bronze

```
64: def carregar_mongodb(...)  
80: montar_filtro_upsert(item)  
82: upsert=True
```

src/database.py:64

SILVER | MongoDB

Coleção `contratacoes_limpas`. Mantém campos aninhados preservados, mas já higienizados para consumo pela aplicação.

Silver

```
93: def carregar_mongodb_limpos(...)  
121: montar_filtro_upsert(item)  
123: upsert=True
```

src/database.py:93

GOLD | MySQL

Tabela `editais_recife`. Recebe apenas colunas escalares, prontas para consulta relacional e análise.

Gold

```
59: def carregar_mysql(...)  
61: engine = create_engine(...)  
62: df.to_sql(...)
```

src/database.py:59

PREFECT: ORQUESTRAÇÃO DO ETL

O orquestrador controla janela mensal, execução das tasks, logs e retentativas.

Janela mensal

```
17: def janela_datas(...)  
30: janela_dias = int(os.getenv(..., '30'))  
34: hoje = datetime.now()  
35: inicio = hoje - timedelta(days=janela_dias)  
36: fim = hoje + timedelta(days=1)
```

orchestrate_prefect.py:17

Flow principal

```
175: def etl_pncp_flow(...)  
185: if data_inicial is None or data_final is None:  
190: run_id = str(uuid.uuid4())  
194: logger.info('PIPELINE INICIADO ...')
```

orchestrate_prefect.py:175

Leitura para apresentação

O Prefect é o maestro do pipeline: calcula datas, chama extração, carrega brutos, transforma, grava limpos e finaliza na camada gold.

Agendamento

O projeto também possui rotina diária via cron às 06:00 em serve_prefect.py e prefect.yaml, evitando execução manual recorrente.

PYSPARK: TRANSFORMAÇÃO EM ESCALA

O Spark aparece como alternativa para processar volumes maiores lendo do MongoDB e escrevendo limpos/gold.

Conexão Spark + Mongo

```
65: def criar_spark_session()
67: SparkSession.builder
73: spark.mongodb.read.connection.uri
75: spark.mongodb.read.collection
81: spark.mongodb.write.collection
```

spark_transformation.py:65

Transformação

```
88: def ler_mongodb(spark)
90: spark.read.format('mongodb').load()
95: def transformar(df)
157: df = df.withColumn('_upsert_key', ...)
190: .option('idFieldList', '_upsert_key')
```

spark_transformation.py:88

Por que Spark?

Quando o volume cresce, o Spark permite transformar dados de forma mais escalável que uma rotina local simples, preservando campos aninhados para o MongoDB.

Saída final

A função salvar_mysql seleciona as colunas relacionais antes de enviar para a tabela editais_recife_spark.

```
223: colunas_relacionais = [c for c in COLUNAS_MYSQL if c in df.columns]
224: df = df.select(colunas_relacionais)
```


SERVIDOR MCP: FERRAMENTAS DE DADOS

O MCP expõe ferramentas controladas para o chatbot consultar dados reais do MongoDB.

Servidor e fonte de dados

```
12: mcp = FastMCP(name='pncp-recife', ...)
17: buscar dados reais exclusivamente do MongoDB
24: MONGO_COLLECTION = ... 'contratacoes_brutas'
27: def _get_mongo_collection():
29: return client[db][collection], client
```

mcp_server.py:12

Ferramentas expostas

```
32: @mcp.tool()
33: def consultar_mongodb(...)
72: def resumir_contratacoes_mongodb()
164: def status_pipeline()
257: mcp.run(transport='stdio')
```

mcp_server.py:32

Papel no sistema

O chatbot não acessa o banco diretamente. Ele pergunta ao MCP, o MCP executa uma ferramenta segura e devolve o resultado para a resposta final.

CHATBOT: INTERFACE STREAMLIT + LLM

A tela em Streamlit conversa com o modelo Groq/Llama e usa function calling para acionar ferramentas do MCP.

Descoberta e execu??o de tools

```
290: async def fetch_mcp_tools()
294: async with ClientSession(...) as session:
296: tools_response = await session.list_tools()
317: async def call_mcp_tool(...)
323: result = await session.call_tool(...)
```

chatbot_app.py:290

Chat com Groq

```
330: async def chat_with_groq(...)
343: client = Groq(api_key=api_key)
354: model = GROQ_MODEL
360: kwargs['tools'] = tools
361: kwargs['tool_choice'] = 'auto'
```

chatbot_app.py:330

Entrada do usu?rio

O Streamlit recebe a pergunta com `st.chat_input` e mant?m hist?rico de conversa na sess?o.

```
549: user_input = st.chat_input(...)
```

Resposta com dados

A LLM decide quando usar uma ferramenta; o resultado do MCP volta para o modelo compor uma resposta em portugu?s sobre contrata??es p?blicas.

```
581: tools = asyncio.run(fetch_mcp_tools())
583: chat_with_groq(active_input, history, tools)
```